

Paservices Automation Integration Guide

This document consolidates everything an automation engineer needs to orchestrate workflows across the **Auth**, **Super ID**, **Data Capture Rightmove**, **Floorplan Analysis**, and **Image Condition Analysis** services. Each service exposes asynchronous pipelines with consistent status notifications (webhooks + S3 snapshots) so you can trigger, monitor, and react to progress and final results.

1. System Overview

Service	Directory	Framework	Purpose
Auth Service	auth_service	FastAPI	Issues machine-to-machine (M2M) tokens for internal services.
Super ID Service	super_id_service	FastAPI	Generates workflow-specific UUIDs.
Data Capture Rightmove Service	data_capture_rightmove_service	FastAPI + Celery	Analyses property URL for property data (combined API)fetches.
Floorplan Analysis Service	floorplan_service	FastAPI + Celery	Analyses property floorplans for scoring and summary insights
Image Condition Analysis Service	(external repo sanalysis)	Django + Celery	Analyses property photos for condition scoring and summary insights.

All services run inside Docker (see `docker-compose.yml`). The `paservices` stack shares Supabase (Postgres) and S3 credentials via `.env` files. The two analysis services (floorplan + image condition) follow the same notification contract:

1. Client submits an async task (`POST /.../analyze` endpoint).
2. Service immediately returns `202` and enqueues work.
3. Background job updates internal DB tables and emits status notifications to S3 and an optional callback URL.
4. Workflow listens to the webhook or pulls the snapshot from S3 to continue the automation. Snapshot here means a saved JSON state of the task at a specific moment in time, stored in S3, representing the progress or results of the async workflow.

2. Authentication + Super IDs

2.1 Auth Service

- **Endpoint:** **POST** `https://auth.supersami.com/api/v1/auth/token`
- **Body:**

```
{
  "grant_type": "client_credentials",
  "client_id": "<m2m-client-id>",
  "client_secret": "<m2m-client-secret>"
}
```

- **Response:**

```
{
  "access_token": "<JWT>",
  "token_type": "bearer",
  "expires_in": 3600
}
```

- Use this token in the **Authorization: Bearer <token>** header for all downstream service calls.
- For testing purposes use this to generate a token:

```
{
  "grant_type": "client_credentials",
  "client_id": "fe2c7655-0860-4d98-9034-cd5e1ac90a41",
  "client_secret": "dev-rightmove-service-secret"
}
```

2.2 Super ID Service

- **Endpoint:** **POST** `https://superid.supersami.com/api/v1/super_ids`
- **Headers:** **Authorization: Bearer <token>**
- **Body:**

```
{
  "count": 1,
  "metadata": {
    "description": "Workflow run for property XYZ"
  }
}
```

- **Response:**

```
{
  "super_id": "50151250-6c58-43f6-a03c-00d5452e35b9"
}
```

3. Data Capture Rightmove Service

3.1 Combined Fetch API

- **Endpoint:** POST `https://data-capture-rightmove.supersami.com/api/v1/properties/fetch/combined`
- **Body:**

```
{
  "super_id": "c2a2f94f-b0d9-4414-99ad-a3e2da1518ab",
  "property_url": "https://www.rightmove.co.uk/properties/154508327#/?channel=RES_LET",
  "callback": {
    "url": "<webhook-base-url>/webhook/status"
  }
}
```

- **Response:** 202 Accepted

```
{
  "message": "Accepted combined fetch request...",
  "success": true
}
```

3.1.1 Mapping scraper JSON to service payloads

When the data capture finishes, it emits a JSON document under `"data_location"`. Watch for `json["status"] == "completed"`. The table below shows where to pull values from that payload when orchestrating downstream services:

Downstream parameter	JSON location	Notes
property_id	json_output["data"] ["property_id"] (integer)	This ID is reused by the floorplan analysis and image-condition anlaysis services for traceability.
property_url	json_output["data"] ["property_url"]	Use when calling the data capture combined fetch endpoint.

Downstream parameter	JSON location	Notes
Bedroom count	json_output["data"]["results"][1] ["raw_data"]["data"] ["bedrooms"]	Optional metadata for image-condition-analysis (<code>notes.bedrooms</code>).
Floorplan details list	json_output["data"]["results"][1] ["raw_data"]["data"] ["floorplans"]	Map each entry to the <code>floorplans</code> dictionary (<code>key -> { "url": ... }</code>).
Property image list	json_output["data"]["results"][1] ["raw_data"]["data"]["images"]	Supply to the image-condition analysis <code>image_urls</code> list.

From the floorplan and image details list, you are interested in the "url" keys from each object. With these mappings in place, a workflow can capture the fields once, store them in the execution context, and feed the same values into all service calls.

3.2 Status Notification

- S3 key: `rightmove/status/fetch_combined/<super_id>/status.json`
- Webhook payload (example):

```
{
  "super_id": "c2a2f94f-b0d9-4414-99ad-a3e2da1518ab",
  "status": "completed",
  "context": "fetch_combined",
  "data_location":
    "https://propertyanalysisstorage.s3.amazonaws.com/rightmove/status/fet
    ch_combined/<super_id>/status.json",
  "summary": {
    "total_endpoints": 2,
    "completed_endpoints": 2
  },
  "metadata": {
    "property_id": 154508327,
    "property_url": "...",
    "client_id": "...",
    "callback_url": "<webhook-base-url>/webhook/status"
  },
  "timestamp": "2025-10-23T10:50:57.757583+00:00"
}
```

- Snapshot (`data.results[]`) contains the raw API responses, success flags, and messages.

4. Floorplan Analysis Service

4.1 API Call

- **Endpoint:** POST `https://floorplan-v1.supersami.com/api/v1/floorplans/analyze`
- **Headers:** Authorization: Bearer <token>
- **Body:**

```
{
  "super_id": "0eae5db5-2cf3-4e82-b6d3-2104a64837fa",
  "property_id": "rightmove-123",
  "floorplans": {
    "fp1": {
      "url": "https://media.rightmove.co.uk/.../FLP_00_0002.jpeg",
      "notes": "Ground floor"
    }
  },
  "callback": {
    "url": "<webhook-base-url>/webhook/status"
  }
}
```

- **Response:** 202 Accepted

```
{
  "message": "Floorplan analysis initiated.",
  "success": true
}
```

You can include **any number of floorplans** in a single request by adding more keys inside the `floorplans` object (for example, `"fp1"`, `"fp2"`, each with its own URL/notes). The service enumerates every key and tracks them individually in the status updates.

Tip: map each floorplan array entry to a deterministic key (e.g., `"fp1"`, `"fp2"`) so that downstream consumers can correlate responses.

4.2 Status Notification

- S3 key: `floorplan/status/floorplan_analyze/<super_id>/status.json`
- Webhook payload:

```
{
  "super_id": "0eae5db5-2cf3-4e82-b6d3-2104a64837fa",
  "status": "completed",
  "context": "floorplan_analyze",
  "summary": { "processed_floorplans": 1, "total_floorplans": 1 },
  "metadata": {
    "property_id": "rightmove-123",
    "callback_url": "...",
    "notes": {}
  }
}
```

```
    },
    "data_location":
    "https://propertyanalysisstorage.s3.amazonaws.com/floorplan/status/...",
    "timestamp": "2025-10-25T07:57:23.616069+00:00"
  }
```

5. Image Condition Analysis Service (Django)

5.1 API Call

- **Endpoint:** POST <https://image-condition-analysis.supersami.com/api/v1/image-condition/analyze>
- **Headers:** Authorization: Bearer <token>
- **Body:**

```
{
  "super_id": "b6964844-25f8-4721-8d20-bb99450d5402",
  "property_id": "rightmove-123",
  "image_urls": [
    "https://media.rightmove.co.uk/.../IMG_00.jpeg",
    "https://media.rightmove.co.uk/.../IMG_01.jpeg"
  ],
  "notes": {
    "bedrooms": 3,
    "floorplan_urls": ["https://...floorplan.jpeg"]
  },
  "callback": {
    "url": "<webhook-base-url>/webhook/status"
  }
}
```

- **Response:** 202 Accepted

```
{
  "super_id": "...",
  "images": 2,
  "status": "queued"
}
```

`property_id` is optional; include it when you want status notifications and downstream storage to reference the same identifier used by floorplan and data-capture workflows.

5.2 Status Notification

- S3 key: `image-condition/status/image_condition_analysis/<super_id>/status.json`
- Webhook payload mirrors Floorplan Analysis Service (status/context/data_location/summary/metadata). Each update includes a `data_location` URL pointing to the live snapshot, and the JSON stored at that URL grows with every stage (downloads, analysis, final result), matching the behaviour of the Data Capture Rightmove and Floorplan Analysis services.
- Snapshot `data` includes:

```
{
  "stage": "complete",
  "message": "Analysis completed",
  "progress": 100.0,
  "stage_progress": {
    "analysis": { "progress": 60.0, ... },
    "overall_analysis": { "progress": 100.0, ... }
  },
  "extra": {
    "final_result": {
      "Condition": {...},
      "Detailed Analysis": {...},
      "Overall Analysis": {...}
    }
  }
}
```

6. Error Handling

- `status === "failed"` payload contains `metadata.error` or `data.extra.error`. Route to Slack/Email nodes for troubleshooting.
 - For partial results (e.g., some floorplans fail), inspect `data.floorplans[].analysis`.
-

7. Summary

- Every long-running task emits **consistent** status notifications (S3 + webhook).
- workflow can:
 - Trigger (`POST /analyze`), include a callback.
 - Listen (Webhook node) for `started/in_progress/completed/failed`.
 - Pull final data (`data_location` URL) for further processing.
- Environment variables control defaults; per-call `callback` overrides are supported.